javascript : événements

événements

Certaines actions sur des éléments d'un document web génèrent un **événement**.
Un événement caractérise l'action réalisée et dépend de l'élément **cible** (sur lequel porte l'action).
Il existe différents types d'événements.

événements

Certaines actions sur des éléments d'un document web génèrent un **événement**.
Un événement caractérise l'action réalisée et dépend de l'élément **cible** (sur lequel porte l'action).
Il existe différents types d'événements.

- actions de l'utilisateur via le clavier ou la souris
  $\longrightarrow$ `click`, `keyup`, `mouseover`, etc.

événements

Certaines actions sur des éléments d'un document web génèrent un **événement**.
Un événement caractérise l'action réalisée et dépend de l'élément **cible** (sur lequel porte l'action).
Il existe différents types d'événements.

- actions de l'utilisateur via le clavier ou la souris
  $\longrightarrow$ `click`, `keyup`, `mouseover`, etc.
- changement d'état
  $\longrightarrow$ `change`, `focus`

événements

Certaines actions sur des éléments d'un document web génèrent un **événement**.
Un événement caractérise l'action réalisée et dépend de l'élément **cible** (sur lequel porte l'action).
Il existe différents types d'événements.

- actions de l'utilisateur via le clavier ou la souris
  $\longrightarrow$ `click`, `keyup`, `mouseover`, etc.
- changement d'état
  $\longrightarrow$ `change`, `focus`
- chargement d'un élément
  $\longrightarrow$ `load`

événements

Certaines actions sur des éléments d'un document web génèrent un **événement**.
Un événement caractérise l'action réalisée et dépend de l'élément **cible** (sur lequel porte l'action).
Il existe différents types d'événements.

- actions de l'utilisateur via le clavier ou la souris
  $\longrightarrow$ `click`, `keyup`, `mouseover`, etc.
- changement d'état
  $\longrightarrow$ `change`, `focus`
- chargement d'un élément
  $\longrightarrow$ `load`
- etc.

programmation événementielle

programmation événementielle

La **programmation événementielle** consiste à lier une fonction à l'occurrence d'un événement sur un élément.

programmation événementielle

programmation événementielle

La **programmation événementielle** consiste à lier une fonction à l'occurrence d'un événement sur un élément.
On parle d'**abonnement** de la fonction à l'élément pour l'événement.

programmation événementielle

programmation événementielle

La **programmation événementielle** consiste à lier une fonction à l'occurrence d'un événement sur un élément.
On parle d'**abonnement** de la fonction à l'élément pour l'événement.
La fonction est déclenchée (exécutée) lorsque l'événement se produit sur cet élément **cible** (***target***).

programmation événementielle

programmation événementielle

La **programmation événementielle** consiste à lier une fonction à l'occurrence d'un événement sur un élément.
On parle d'**abonnement** de la fonction à l'élément pour l'événement.
La fonction est déclenchée (exécutée) lorsque l'événement se produit sur cet élément **cible** (***target***).

fonction *listener*

La fonction attachée à un événement est appelée fonction « gestionnaire d'événement » – *event handler* – ou « d'écoute » – ***event listener***.

méthode d'abonnement

addEventListener

La méthode `addEventListener` réalise l'abonnement d'une fonction à un événement donné pour l'objet sur lequel elle est invoquée .

objet.`addEventListener(`*eventType*`,` *listenerFunction*`)`

méthode d'abonnement

addEventListener

La méthode `addEventListener` réalise l'abonnement d'une fonction à un événement donné pour l'objet sur lequel elle est invoquée .

objet.`addEventListener(`*eventType*`,` *listenerFunction*`)`

- *objet* : l'objet ciblé : `window`, ou un élément de la page

méthode d'abonnement

addEventListener

La méthode `addEventListener` réalise l'abonnement d'une fonction à un événement donné pour l'objet sur lequel elle est invoquée .

objet.`addEventListener(`*eventType*`,` *listenerFunction*`)`

- *objet* : l'objet ciblé : `window`, ou un élément de la page
- *eventType* : une chaîne de caractères désignant l'événement concerné
  `"click"`, `"load"`, `"change"`, `"mouseover"`, `"keypress"` etc.

méthode d'abonnement

addEventListener

La méthode `addEventListener` réalise l'abonnement d'une fonction à un événement donné pour l'objet sur lequel elle est invoquée .

objet.`addEventListener(`*eventType*`,` *listenerFunction*`)`

- *objet* : l'objet ciblé : `window`, ou un élément de la page
- *eventType* : une chaîne de caractères désignant l'événement concerné
  `"click"`, `"load"`, `"change"`, `"mouseover"`, `"keypress"` etc.
- *listenerFunction* : la fonction *listener* qui est appelée lorsque l'événement se produit

attention

lors d'un abonnement

objet.`addEventListener(`*eventType*`,` *listenerFunction*`)`

listenerFunction est une **valeur** de type fonction,
ce n'est pas l'appel de la fonction

```
var action1 = function() {
   window.alert("on a cliqué sur le pied de page");
}

     // sélection d'un élément
var pied =  document.getElementById("piedDePage");
     // abonnement sur cet élément de la fonction 'action1'
     // pour un événement "click" => 'action()' déclenchée si clic
pied.addEventListener("click",action1);
```

exemple_event1.html

un peu de méthodologie

- où placer les définitions des fonctions listeners ?

un peu de méthodologie

- où placer les définitions des fonctions listeners ?
- quand et où faire les abonnements ?

un peu de méthodologie

- où placer les définitions des fonctions listeners ?
- quand et où faire les abonnements ?

suggestion méthodologique :

exemple_event2.html – *temperature.html*

un peu de méthodologie

- où placer les définitions des fonctions listeners ?
- quand et où faire les abonnements ?

suggestion méthodologique :

1. placer les fonctions javascript dans un fichier à part de l'html

exemple_event2.html – *temperature.html*

un peu de méthodologie

- où placer les définitions des fonctions listeners ?
- quand et où faire les abonnements ?

suggestion méthodologique :

1. placer les fonctions javascript dans un fichier à part de l'html
2. définir une fonction `setupListeners` chargée de mettre en place les abonnements :

exemple_event2.html – *temperature.html*

un peu de méthodologie

- où placer les définitions des fonctions listeners ?
- quand et où faire les abonnements ?

suggestion méthodologique :

1. placer les fonctions javascript dans un fichier à part de l'html
2. définir une fonction `setupListeners` chargée de mettre en place les abonnements :
   1. récupérer l'élément ciblé

exemple_event2.html – *temperature.html*

un peu de méthodologie

- où placer les définitions des fonctions listeners ?
- quand et où faire les abonnements ?

suggestion méthodologique :

1. placer les fonctions javascript dans un fichier à part de l'html
2. définir une fonction `setupListeners` chargée de mettre en place les abonnements :
   1. récupérer l'élément ciblé
   2. abonner la fonction listener pour l'événement voulu

exemple_event2.html – *temperature.html*

un peu de méthodologie

- où placer les définitions des fonctions listeners ?
- quand et où faire les abonnements ?

suggestion méthodologique :

1. placer les fonctions javascript dans un fichier à part de l'html
2. définir une fonction `setupListeners` chargée de mettre en place les abonnements :
   1. récupérer l'élément ciblé
   2. abonner la fonction listener pour l'événement voulu
3. déclencher la fonction `setupListeners`

exemple_event2.html – *temperature.html*

un peu de méthodologie

- où placer les définitions des fonctions listeners ?
- quand et où faire les abonnements ?

suggestion méthodologique :

1. placer les fonctions javascript dans un fichier à part de l'html
2. définir une fonction `setupListeners` chargée de mettre en place les abonnements :
   1. récupérer l'élément ciblé
   2. abonner la fonction listener pour l'événement voulu
3. déclencher la fonction `setupListeners`
   - doit être fait quand les éléments existent, donc après leur création

exemple_event2.html – *temperature.html*

un peu de méthodologie

- où placer les définitions des fonctions listeners ?
- quand et où faire les abonnements ?

suggestion méthodologique :

1. placer les fonctions javascript dans un fichier à part de l'html
2. définir une fonction `setupListeners` chargée de mettre en place les abonnements :
   1. récupérer l'élément ciblé
   2. abonner la fonction listener pour l'événement voulu
3. déclencher la fonction `setupListeners`
   - doit être fait quand les éléments existent, donc après leur création
   - lorsque la page est complètement chargée
     ↪ utilisation de l'événement `load` sur `window`

exemple_event2.html – *temperature.html*

```
<html>
  <head> ...
     <script src="monscript.js"></script>
  </head>
  <body> ...
     <img id="monImage" ... /> ...
     <div id="piedDePage"> ... </div>
     ...
```

exemple_event2.html

temperature.html

avec `monscript.js` :

```
                        // fonction de mise en place des abonnements
var setupListeners = function() {
                        // abonnement pour élément d'id "piedDePage"
  var pied = document.getElementById("piedDePage");
  pied.addEventListener("click",action1);
                        // abonnement pour élément d'id "monImage"
  var image = document.getElementById("monImage");
  image.addEventListener("mouseover",action2);
}
        // on provoque l'exécution de "setupListeners" à la fin du
        //                chargement du document
window.addEventListener("load", setupListeners);
        // définition des fonctions d'écoute (listener)
var action1 = function() { ... }
var action2 = function() { ... }
```

this

- dans une fonction *listener*, la variable `this` est définie et désigne l'objet **cible** de l'événement
  typiquement un élément du document

temperature.html

this

- dans une fonction *listener*, la variable `this` est définie et désigne l'objet **cible** de l'événement
  typiquement un élément du document

temperature.html

- permet d'utiliser la même fonction listener sur plusieurs éléments en contextualisant son action sur l'élément cible

exemple_this.html

Sur un élément donné on peut avoir

- plusieurs abonnements pour différents événements
- plusieurs abonnements pour le même événement

Sur un élément donné on peut avoir

- plusieurs abonnements pour différents événements
- plusieurs abonnements pour le même événement

removeEventListener

la méthode `removeEventListener` permet de désabonner de l'objet sur lequel elle est invoquée une fonction pour un événement
objet.`removeEventListener(`*eventType*`,` *listenerFunction*`)`

exemple_event3.html

l'objet `event`

- un objet **`event`** est créé pour chaque événement.

l'objet `event`

- un objet `event` est créé pour chaque événement.
- cet objet est passé en paramètre à la fonction listener associée lors de son appel

l'objet `event`

- un objet `event` est créé pour chaque événement.
- cet objet est passé en paramètre à la fonction listener associée lors de son appel
- le type d'objet `event` varie selon l'événement.

l'objet `event`

- un objet `event` est créé pour chaque événement.
- cet objet est passé en paramètre à la fonction listener associée lors de son appel
- le type d'objet `event` varie selon l'événement.
- un objet `event` possède des propriétés qui informent sur l'événement.

l'objet `event`

Quelques propriétés (selon les types d'événements) :

- `type` le type de l'événement,

exemple_event4.html *exemple_target.html*

l'objet `event`

Quelques propriétés (selon les types d'événements) :

- `type` le type de l'événement,
- `clientX`, `clientY` – `screenX`, `screenY` – `pageX`, `pageY` coordonnées de l'événement par rapport au "navigateur" – l'écran – la page

exemple_event4.html *exemple_target.html*

l'objet event

Quelques propriétés (selon les types d'événements) :

- `type` le type de l'événement,
- `clientX`, `clientY` – `screenX`, `screenY` – `pageX`, `pageY` coordonnées de l'événement par rapport au "navigateur" – l'écran – la page
- `altKey`, `ctrlKey`, `shiftKey` l'une des touches Alt, Ctrl ou Shift était pressée lors de l'événement ?

exemple_event4.html *exemple_target.html*

l'objet event

Quelques propriétés (selon les types d'événements) :

- `type` le type de l'événement,
- `clientX`, `clientY` – `screenX`, `screenY` – `pageX`, `pageY` coordonnées de l'événement par rapport au "navigateur" – l'écran – la page
- `altKey`, `ctrlKey`, `shiftKey` l'une des touches Alt, Ctrl ou Shift était pressée lors de l'événement ?
- `key` information sur la touche appuyée

exemple_event4.html *exemple_target.html*

l'objet event

Quelques propriétés (selon les types d'événements) :

- `type` le type de l'événement,
- `clientX`, `clientY` – `screenX`, `screenY` – `pageX`, `pageY` coordonnées de l'événement par rapport au "navigateur" – l'écran – la page
- `altKey`, `ctrlKey`, `shiftKey` l'une des touches Alt, Ctrl ou Shift était pressée lors de l'événement ?
- `key` information sur la touche appuyée
- `target` la cible de l'événement (== `this`)

exemple_event4.html *exemple_target.html*

l'objet event

Quelques propriétés (selon les types d'événements) :

- `type` le type de l'événement,
- `clientX`, `clientY` – `screenX`, `screenY` – `pageX`, `pageY` coordonnées de l'événement par rapport au "navigateur" – l'écran – la page
- `altKey`, `ctrlKey`, `shiftKey` l'une des touches Alt, Ctrl ou Shift était pressée lors de l'événement ?
- `key` information sur la touche appuyée
- `target` la cible de l'événement (== `this`)
- `timestamp` le moment de création de l'événement

exemple_event4.html *exemple_target.html*

l'objet event

Quelques propriétés (selon les types d'événements) :

- `type` le type de l'événement,
- `clientX`, `clientY` – `screenX`, `screenY` – `pageX`, `pageY` coordonnées de l'événement par rapport au "navigateur" – l'écran – la page
- `altKey`, `ctrlKey`, `shiftKey` l'une des touches Alt, Ctrl ou Shift était pressée lors de l'événement ?
- `key` information sur la touche appuyée
- `target` la cible de l'événement (== `this`)
- `timestamp` le moment de création de l'événement
- etc.

exemple_event4.html *exemple_target.html*

versions précédentes

- DOM niveau 0 :
  - les gestionnaires d'événements s'appellent `onevent` : `onclick`, `onload`, `onmmouseover`, etc.
  - un gestionnaire d'événement peut être placé *en ligne* en tant qu'attribut d'un élément, la valeur associée est le code exécuté

```
<img src="..." onclick="var i = 2; action();"/>
```

  - on peut aussi créer l'abonnement dans le code javascript :

```
element.onclick = maFonction;
```

- défauts :
  - code javascript dans le code HTML
  - un seul abonnement possible par type d'événement

Conclusion

utiliser le modèle `addEventListener`

- `attachEvent(...)` dans IE < 8 (+ autres différences)

à suivre...

javascript : manipuler DOM